MedDecode

Input & Output Data Contract

Between Member 1 — Intelligence Service and Member 2 — Backend/Application Service

Version: 1.0

Date: 6 September 2026

Status: Initial working contract for parallel development

Purpose

This document defines the technical contract between the MedDecode Intelligence component (Member 1) and the MedDecode Backend/Application component (Member 2). Its purpose is to allow both members to start development independently without tightly coupling internal implementations. Member 2 owns user-facing application flow and persistence. Member 1 receives a prescription document, performs prescription intelligence, and returns a complete structured JSON result.

Core rule: Member 2 should treat Member 1 as a black-box Prescription Intelligence Service. Member 1 may change internal OCR, datasets, matching models, or ranking methods without requiring Member 2 changes, provided this contract remains backward compatible.

1. Responsibility Boundary

Member 2 — Backend/Application	Member 1 — Intelligence
Accepts user uploads	Receives prescription image/PDF or authorized document reference
Authentication and authorization	Document processing and OCR/vision
AWS S3 upload/storage and access control	Extracts available prescription details
Creates prescription/job record	Extracts medicine and dosage information
Calls Intelligence Service	Normalizes extracted information
Stores returned JSON/results	Matches medicines against intelligence dataset
Prescription history and user history	Returns ingredients/salts and medicine intelligence
Pill tracking, analytics and dashboard	Generates recommendations and eligible alternatives
Displays results to user	Returns confidence, warnings and processing metadata

2. End-to-End Flow

```
USER
  |
  | uploads prescription image/PDF
  v
MEMBER 2 BACKEND
  |
  | creates prescription_id + job_id
  | stores file / prepares authorized access
  v
MEMBER 1 PRESCRIPTION INTELLIGENCE SERVICE
  |
  | OCR / vision
  | extraction
  | medicine matching
  | ingredient intelligence
```

```
  | recommendations / alternatives
  v
FINAL INTELLIGENCE JSON
  |
  v
MEMBER 2 BACKEND
  |
  | validate + persist + display
  v
HISTORY / TRACKING / ANALYTICS / DASHBOARD
```

3. API Interaction Model

The initial integration should support one request that submits a prescription for analysis and one structured response. The actual transport mechanism may be HTTP/REST or another agreed internal service interface. The JSON schemas below are the contract; endpoint naming can be finalized during implementation.

Recommended logical operation: **Analyze Prescription**

4. Member 2 → Member 1: Input Contract

Member 2 sends one analysis request per prescription processing job.

```
{
  "schema_version": "1.0",
  "job_id": "UUID - required",
  "prescription_id": "UUID - required",
  "document": {
    "access_type": "authorized_url | direct_upload",
    "file_url": "temporary authorized URL when access_type is authorized_url",
    "file_type": "image/jpeg | image/png | application/pdf",
    "original_filename": "optional"
  },
  "requested_analysis": {
    "extract_prescription": true,
    "match_medicines": true,
    "find_alternatives": true,
    "generate_recommendations": true
  }
}
```

4.1 Input Field Rules

Field	Required	Rule
schema_version	Yes	Contract version used by caller.
job_id	Yes	Unique processing request identifier. Used for idempotency/tracking.
prescription_id	Yes	Backend-owned identifier for the prescription record.
document	Yes	Exactly one usable prescription document must be supplied.
document.access_type	Yes	Defines how Member 1 accesses the document.
document.file_url	Conditional	Required when authorized_url is used. Must be valid long enough for processing.
document.file_type	Yes	MIME type declared by Member 2.
document.original_filename	No	Optional diagnostic metadata.
requested_analysis	Yes	Feature flags. Member 2 can disable optional analysis if required.

4.2 Input Validation Requirements

• Member 2 must generate job_id and prescription_id before calling Member 1.

• Member 1 must reject unsupported or inaccessible documents with a structured error response.

• Member 1 must not assume that every prescription contains the same fields.

• No patient details need to be supplied separately for v1. Extractable patient/doctor/facility information is read from the document.

• Missing information must not be invented by either service.

5. Member 1 → Member 2: Output Contract

Member 1 returns the complete intelligence result. Member 2 persists the information needed for product features such as prescription history, medicine history, pill tracking, analytics, and user display.

```
{
  "schema_version": "1.0",
  "job_id": "UUID",
  "prescription_id": "UUID",
  "status": "completed",
  "prescription": {
    "prescription_date": null,
    "doctor": {
      "name": null,
      "registration_number": null
    },
    "facility": {
      "name": null,
      "address": null,
      "phone": null
    }
  },
  "patient": {
    "name": null,
    "age": null,
    "gender": null
  },
  "medicines": [],
  "recommendations": [],
  "warnings": [],
  "processing_metadata": {
    "overall_confidence": null,
    "dataset_version": "1.0",
    "model_version": "1.0"
  }
}
```

5.1 Top-Level Output Rules

Field	Required	Description
schema_version	Yes	Output contract version.
job_id	Yes	Must equal the input job_id.
prescription_id	Yes	Must equal the input prescription_id.
status	Yes	completed | completed_with_warnings | failed.
prescription	Yes	Object; fields may be null when absent/unextractable.
patient	Yes	Object; fields may be null when absent/unextractable.
medicines	Yes	Array; empty when no medicine could be reliably extracted.
recommendations	Yes	Array; may be empty.
warnings	Yes	Array; may be empty.
processing_metadata	Yes	Confidence and intelligence version metadata.

6. Prescription-Level Extraction Contract

```
{
  "prescription": {
    "prescription_date": "YYYY-MM-DD or null",
    "doctor": {
      "name": "string or null",
      "registration_number": "string or null"
    },
    "facility": {
      "name": "string or null",
      "address": "string or null",
      "phone": "string or null"
    }
  },
  "patient": {
    "name": "string or null",
    "age": "integer or null",
    "gender": "string or null"
  }
}
```

Important: These values are document-extracted values. Member 2 must not automatically overwrite authenticated user profile information with extracted document data.

7. Per-Medicine Output Contract


```
{
  "sequence_number": 1,
  "extraction": {
    "raw_text": "Tab Dolo 650 1-0-1 AF for 5 days",
    "medicine_name": "Dolo 650",
    "medicine_name_normalized": "dolo 650",
    "strength": {
      "value": 650,
      "unit": "mg"
    },
    "dosage_form": "tablet",
    "frequency": {
      "type": "scheduled",
      "morning": 1,
      "afternoon": 0,
      "night": 1,
      "times_per_day": 2
    },
    "timing": "after_food",
    "duration": {
      "value": 5,
      "unit": "days"
    },
    "quantity": {
      "value": null,
      "unit": null
    },
    "instructions": null
  },
  "confidence": {
    "extraction": 0.95,
    "medicine_name": 0.98,
    "dosage": 0.94
  },
  "medicine_match": {
    "status": "matched",
    "confidence": 0.97,
    "medicine_external_id": "stable identifier owned by Intelligence",
    "brand_name": "Dolo 650",
    "dosage_form": "tablet",
    "ingredients": [
      {
        "name": "Paracetamol",
        "strength_value": 650,
        "strength_unit": "mg"
      }
    ]
  },
  "derived": {
    "units_per_day": 2,
    "estimated_total_units": 10
  },
  "alternatives": []
}
```

8. Medicine Field Definitions

Field	Mandatory in object	Meaning / missing-data behavior
sequence_number	Yes	Position of the medicine in the extracted prescription. Starts at 1.
extraction.raw_text	Yes	Source medicine instruction text used by the intelligence pipeline.
medicine_name	No	Extracted name; null when not recoverable.
medicine_name_normalized	No	Normalized extraction; null when no reliable name exists.
strength.value/unit	No	Both may be null if strength is absent.
dosage_form	No	Normalized form such as tablet/capsule/syrup; null if unknown.
frequency	Yes	Object. Individual schedule values may be null.
frequency.type	Yes	scheduled | as_needed | unknown.
timing	No	Example: before_food, after_food; null if absent.
duration	Yes	Object. value/unit null when absent.
quantity	Yes	Object. value/unit null when absent.
instructions	No	Additional instruction text when extractable.
confidence	Yes	Confidence values in the range 0 to 1, or null when not calculated.
medicine_match	Yes	Match result object; detailed fields depend on status.
derived	Yes	Derived analytics values; null when insufficient information.
alternatives	Yes	Array; empty when none are eligible/available.

9. Medicine Match Contract

```
{
  "medicine_match": {
    "status": "matched | probable_match | ambiguous | unmatched | not_attempted",
    "confidence": 0.0,
    "medicine_external_id": "string or null",
    "brand_name": "string or null",
    "dosage_form": "string or null",
    "ingredients": [
      {
        "name": "string",
        "strength_value": 0,
        "strength_unit": "string"
      }
    ]
  }
}
```

Status interpretation:

• **matched:** Intelligence engine identified a sufficiently confident medicine record.

• **probable_match:** A likely candidate exists but confidence is below the strict match threshold.

• **ambiguous:** More than one candidate is plausible and no safe single choice should be asserted.

• **unmatched:** No acceptable medicine record was identified.

• **not_attempted:** Matching was disabled or could not be attempted.

10. Derived Analytics Contract

```
{
  "derived": {
    "units_per_day": "number or null",
    "estimated_total_units": "number or null"
  }
}
```

Member 1 may calculate units_per_day only when sufficient structured dosage and frequency information exists. For as-needed/SOS medicines, frequency-based derived values should normally remain null. estimated_total_units should

only be returned when both daily units and duration are sufficiently known. No averages or assumptions should be invented.

11. Alternatives Contract

```
{
  "alternatives": [
    {
      "medicine_external_id": "string",
      "brand_name": "string",
      "match_basis": {
        "ingredient_equivalence": true,
        "strength_compatibility": true,
        "dosage_form_compatibility": true
      },
      "price": {
        "value": null,
        "currency": null,
        "unit_description": null
      },
      "confidence": 0.0,
      "note": "string or null"
    }
  ]
}
```

An alternative must not be presented by the Intelligence Service unless its compatibility criteria are satisfied according to the configured intelligence rules. Price fields are optional because reliable pricing data may not be available in every dataset.

12. Recommendations and Warnings Contract

```
{
  "recommendations": [
    {
      "type": "string",
      "title": "string",
      "message": "string",
      "related_medicine_sequence_number": null,
      "confidence": null
    }
  ],
  "warnings": [
    {
      "code": "string",
      "severity": "info | low | medium | high",
      "message": "string",
      "related_medicine_sequence_number": null
    }
  ]
}
```

Recommendations and warnings are intelligence outputs, not substitutes for clinical judgment. Member 2 should display them according to the product's approved UX and safety rules and should preserve the warning severity and message without silently changing their meaning.

13. Missing, Unknown and Uncertain Data Rules

Situation	Contract behavior
Field absent from prescription	Return null for the field.
Array has no items	Return an empty array [].
Object is part of stable schema but values unavailable	Return the object with its defined fields set to null where appropriate.
OCR/extraction uncertain	Return extracted value only if available and include confidence when calculated; otherwise use null.
Medicine cannot be matched	medicine_match.status = unmatched and medicine identifiers/details are null or empty according to schema.
Processing cannot complete	Return status = failed with structured error object.
Do not know	Never invent values such as Unknown/N/A as data values; use null unless the field itself is an enumerated status.

14. Processing Status and Error Contract

```
{
  "schema_version": "1.0",
  "job_id": "UUID",
  "prescription_id": "UUID",
  "status": "failed",
  "error": {
    "code": "UNSUPPORTED_FILE | DOCUMENT_ACCESS_FAILED | INVALID_REQUEST | PROCESSING_FAILED",
    "message": "human-readable summary",
    "retryable": true
  }
}
```

Member 2 must use job_id to correlate responses with the original request. Member 1 must return the same identifiers supplied in the request whenever a response can be generated.

15. Versioning and Backward Compatibility

• Both request and response include schema_version.

• Version 1.0 must remain stable while Member 2 integrates.

• Member 1 may add new optional fields in a backward-compatible update.

• Member 1 must not rename, remove, or change the meaning/type of an existing required field without a new schema version.

• Member 2 should ignore unknown optional fields so newer intelligence versions do not break the backend.

• processing_metadata should carry dataset_version and model_version when available for traceability.

16. Ownership of Stable Identifiers

job_id and prescription_id are owned by Member 2. Member 1 echoes them back and does not redefine them. **medicine_external_id is owned by Member 1.** It is a stable public identifier for a canonical medicine record and must not expose internal database implementation details. Member 2 may store this identifier for linking returned results to medicine intelligence, but should not assume access to Member 1's internal database schema.

17. Final Integration Checklist

1. Member 2 can upload JPG/JPEG/PNG/PDF and create prescription_id + job_id.
2. Member 2 can provide Member 1 authorized document access.
3. Member 1 can validate the request and access the document.
4. Member 1 can return status and the same correlation IDs.
5. Member 1 can return null for absent fields without breaking schema.
6. Member 1 can return multiple medicines in sequence_number order.
7. Member 1 can return match status and confidence independently of extraction.
8. Member 2 can store the returned result and preserve medicine_external_id.
9. Member 2 can build history/tracking/analytics from structured medicine and derived fields.
10. Both members can continue internal development without sharing internal database implementation.

Contract Summary

```
MEMBER 2 → MEMBER 1
Prescription image/PDF
+ job_id
+ prescription_id
+ document access metadata
+ requested analysis flags

MEMBER 1
Processes the entire prescription:
OCR / vision
→ information extraction
→ medicine extraction
→ normalization
→ medicine matching
→ ingredient intelligence
→ derived values
→ recommendations
→ eligible alternatives
→ warnings/confidence

MEMBER 1 → MEMBER 2
One complete, versioned Prescription Intelligence JSON result.

MEMBER 2
Validates, stores, displays, tracks history, and generates user-facing analytics.
```